

Speaker Notes

Contents

- [1 1. I/O from Python to the operating system](#)
- [2 2. Roadmap](#)
- [3 3. Takeaways](#)
- [4 4. What this lecture covers](#)
- [5 5. What this lecture leaves out](#)
- [6 6. What does one byte cost?](#)
- [7 7. What does one byte cost?](#)
- [8 8. Two programs, two access patterns](#)
- [9 9. Predict before we run](#)
- [10 10. The measurement runs here, not on a slide](#)
- [11 11. Your laptop will not agree with this projector](#)
- [12 12. Where does the difference come from?](#)
- [13 13. Where does the difference come from?](#)
- [14 14. What one crossing costs](#)
- [15 15. What one trace line says](#)
- [16 16. Count the crossings](#)
- [17 17. The count ratio over-predicts the time](#)
- [18 18. What a buffer actually does](#)
- [19 19. Why a seek gives the saving back](#)
- [20 20. Does the seek itself cost a crossing?](#)
- [21 21. Trust the counts. Do not trust the times.](#)
- [22 22. Windows is absent here, on purpose](#)
- [23 23. It is not one buffer — it is a stack](#)
- [24 24. It is not one buffer — it is a stack](#)
- [25 25. Four levels, and who empties each one](#)
- [26 26. To flush is to cross one boundary](#)
- [27 27. Wholesale down, retail up](#)
- [28 28. Python's buffer does not sit on the C library's](#)
- [29 29. The bottom level acknowledges early too](#)
- [30 30. Three lines that print nothing](#)
- [31 31. Why the file is empty](#)
- [32 32. What this demonstration does not prove](#)
- [33 33. Which levels can you actually watch?](#)

34 34. The same write, at three levels

35 35. The same write, at three levels

36 36. Meet the program: `iodemo.c`

37 37. Three calls in the source. One in the trace.

38 38. `fwrite` takes a size and a count

39 39. Two operating systems, side by side

40 40. Why NtTrace and not Process Monitor

41 41. Three Windows words you will need

42 42. A real Windows trace

43 43. A handle is not a file descriptor

44 44. Two error namespaces in one return value

45 45. The stable boundary sits at a different level

46 46. Where the boundary sits is contested history

47 47. Linux is the outlier, not Windows

48 48. What that trace cannot do

49 49. What we have not looked at

50 50. `write()` returned success. The bytes are not on a disk.

51 51. Buffering moves your errors

52 52. Never retry a failed close

53 53. Networking and IPC are this same mechanism

54 54. Honest limits

55 55. Honest limits, continued

56 56. What to do on Monday

57 57. Summary — what follows

58 58. References

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Slides: 58 · **Duration:** 62 min · **Venue:** Faculty of Informatics, Masaryk University (FI MUNI)

11. I/O from Python to the operating system

On screen:

PB158 · Lecture 2

The call you write and the call the operating system runs are not the same call [inference]. `fflush()` flushes only the C library's buffers, and the kernel buffers need a second flush [S3]. Three of the four boundaries below your code have a tool that watches them [V10] [S1] [V30]. No tool in this lecture watches the fourth [V5].

*Ondrej (Ondra) Krajicek me@ondrejkrjicek.com · www.linkedin.com/in/OndrejKrajicek
Built at render time*

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

Welcome back. Lecture one was about how this course works. This one is the first technical lecture and it runs for about an hour.

Let me set the frame in one sentence. You write Python and you write C. Your theory course told you what a system call is. Most of you have never watched one happen [speculative]. That is the gap this lecture closes, and it closes it with a measurement rather than a definition.

Three programs run live on this machine. If a demonstration fails, every claim I rely on is also written on a slide. Nothing in the argument depends on the projector.

One housekeeping note. Everything written in this course is in English, including the slides and the code comments. I speak Czech. The written English is deliberately plain so you can read it while listening to me.

2 2. Roadmap

On screen:

Roadmap

1

What does one byte cost?

A stopwatch.

2

Where does the difference come from?

A tracer.

3

It is not one buffer — it is a stack

A diagram.

4

The same write, at three levels

A program.

5

Two operating systems, side by side

Two traces.

6

What we have not looked at

One open question.

Here is the shape of the hour. I want to tell you where the payoff sits so you can wait for it.

Sections one and two are twenty minutes on a single question. Why do two programs that write the same bytes take very different times? The answer is a number you can count, and by the end of section two you will count it yourself.

Section three is the structural part. I draw one diagram there. I point back at that diagram in the durability lecture, in the memory mapping lecture and in the crash analysis lecture.

Sections four and five spend the tracer. First on one program at three levels, then on two operating systems.

Section six deliberately leaves one question open. That question is the bridge into the rest of the course.

Assignment one measures what section one measures. You will have the model to explain your own numbers.

3 3. Takeaways

On screen:

Where we are going

1

Buffering pays on adjacency

A seek forces the buffer out [inference].

2

A library call is not a system call

Several levels sit beneath yours [S3].

3

The boundary is observable

A tracer shows what your line became [S1].

4

The platforms differ by design

Each nominates a different level [V8].

These four claims are what I want you to hold in a week. I show them now so you know what each section is for.

Number one is the one you use on Monday, and it is what assignment one measures. Notice that it is not the rule "buffering is fast". It has a condition in it, and the condition is the interesting part. Lampson named sequential access beside his batching advice in 1983 [S12].

Number two is the structural claim. It is why this lecture exists.

Number three is the promise about the instrument. By the end of section two you will read a trace line. You will also know which parts of it to believe.

Number four is the platform claim. Some of you run Windows. I am not going to tell you that one system is right. I am going to show you that they nominate different levels as stable.

I bring these back at the end, and each section earns one of them.

4 4. What this lecture covers

On screen:

Scope

1

Why two programs differ

2

When a buffer helps

3

The four levels

4

Reading a trace

What is on this slide is what assignment one measures. Everything I leave out comes on the next slide, with a lecture number attached.

5 5. What this lecture leaves out

On screen:

Scope

5

When a write is safe

6

Memory mapping

7

Operating the tools

8

Asynchronous I/O

I want to fence the hour, because three of these exclusions are the ones you will most want to ask about.

The first is durability. I will show you today that "a successful return from write() does not make any guarantee that data has been committed to disk" [S2]. I will not tell you today what does, because today's instrument cannot answer it. Today's trace stops at the system call boundary, and no trace separates a device that flushed from one that only reported it [V5]. The durability lecture answers the question with a different method.

The second is running the tools. You have no toolchain installed yet. Lecture one told you to install it before the first lab. So today you read traces and I run them. Reading a trace and producing one are different skills.

The third is asynchronous I/O. It changes the picture, and I will say one sentence about it near the end.

Everything here has a lecture number attached, and I will name each one as we pass it.

6 6. What does one byte cost?

On screen:

Part I

Two programs. The same mebibyte. Predict the ratio before we run them.

Nine minutes, and this part contains no theory at all. It contains two programs and a stopwatch.

The order is deliberate. The usual way to teach this is to explain the buffer first and then show a benchmark that confirms it. That teaches you to trust the explanation. I want you to see the number first, decide you cannot explain it, and then want the explanation.

Write your prediction down. Do not keep it in your head.

7 7. What does one byte cost?

On screen:

Part I

Two programs write the same mebibyte to a file. That is 1,048,576 bytes. One uses a buffer. One does not.

[code]

One argument differs. Python documents it: "Pass 0 to switch buffering off (only allowed in binary mode)" [S6].

[S6] Built-in Functions, open() — Python documentation. <https://docs.python.org/3/library/functions.html#open>

Read both programs now. The next twenty minutes depend on you believing they are the same program apart from one argument [S6].

Same loop. Same iteration count. Same interpreter. Same bytes, in the same order, to the same kind of file.

Notice also that you have to ask for the unbuffered version. It is not the default. You cannot write program A by accident, and that will matter when we decide what the practical lesson is.

One word about the word buffer, because I use it in exactly one sense today. A buffer is memory that collects bytes for consecutive positions in the file. The next level then takes them in one call instead of many. It saves calls. It does not save work.

8 8. Two programs, two access patterns

On screen:

Part I

Sequential access

Each write continues where the last one ended [inference].

Random access

Each write follows a move to a new offset [inference].

Why it matters

Lampson advised batch processing in 1983. He named sequential access in the next sentence, as a second and separate reason [S12].

[S12] Hints for Computer System Design — Lampson (1983), p. 19. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>

Two terms, and I hold them to these meanings for the rest of the course [speculative].

Sequential access means each write continues where the last one ended. Random access means each write follows a move to another offset. Do not say scattered and do not say non-linear. Say random access, because that is what the assignment says.

Both programs on the last slide use sequential access. Assignment one asks you to run the comparison a second time with random access and explain the difference. I tell you now so you can listen to section two with the assignment in mind.

The vocabulary is not new. Lampson wrote both terms down in 1983, in a paper of design hints. Use batch processing if you can, he says. Doing things incrementally almost always costs more. He then adds that disks and tapes work much better when accessed sequentially. Read that clause carefully. He sets it aside from the batching advice. He does not rest the advice on it.

That sentence is forty-three years old, and it names the two access patterns you measure next week.

9 9. Predict before we run

On screen:

Part I

Write down a ratio. How much slower is the unbuffered program?

1×

nothing worth measuring

10×

an optimisation

100×

a different kind of thing

Writing a number down before the run changes what you do with the result. Taber calls the method predict-observe-explain, and reports that "making a prediction will give students some sense of ownership in the following activity" [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Pick one of the three numbers. Write it on paper. Take thirty seconds [S14].

I will ask for a show of hands, and I want you to decide before you see the answer. This is not a game. The method has a name. Science teaching uses it for this case, where a room holds a strong intuition. The machine does not match that intuition.

Hands up for one times. Hands up for ten times. Hands up for one hundred.

Now I run both programs on this machine and we see which hand was closest.

Whatever number appears is a fact about this machine, this Python version and this filesystem. Your number will differ, and that turns out to be the lesson rather than a nuisance.

10 10. The measurement runs here, not on a slide

On screen:

Part I

I print no ratio here, on purpose. A benchmark you did not watch me produce is the kind of claim this course teaches you to distrust.

Held fixed

Loop, interpreter, iteration count, bytes, filesystem [inference].

Varied

One argument, buffering=0 [S6].

Write down

Both times, the Python version, the buffer size [inference].

What it includes

The operating system's own memory for file data, in both runs. Name the cache layer your number includes, because you cannot remove all of them [V19].

I am running both programs now, untraced, with a plain wall-clock timer.

While they run, let me be honest about what this measures. One mebibyte does not reach a disk in any way you could detect. It lands in the operating system's own memory and stays there. So this is not a measurement of a disk. It is a measurement of the cost of asking the operating system to do something, a million times against a few hundred. Name the cache layer your number includes, because you cannot remove all of them [V19].

Copy both times into your notebook. You need them for assignment one, and the review lecture on the sixth of October uses your numbers rather than mine.

Now look at the ratio. Then look at the hands that went up. For most rooms the honest reaction is that this stopped being an optimisation. That reaction is the whole job of part one.

11 11. Your laptop will not agree with this projector

On screen:

Part I

The buffer size is not a constant. It grew sixteen-fold in Python 3.14.

Python 3.13

The default "was set to 8192 since 16 years ago" [S7].

Python 3.14

"the buffer will typically be 128 kilobytes long" [S6].

Why

Write performance improved "3 to 5 times" over the 8192-byte default [S7]. The reporter measured it. No independent reproduction [inference].

So

Report your version and buffer size with every number [inference].

[S6] Python documentation, `open()`. [S7] CPython issue 117151. <https://github.com/python/cpython/issues/117151>

This slide is why I refuse to print a benchmark figure [speculative].

Python's default buffer was eight kilobytes for sixteen years. It is now typically one hundred and twenty-eight kilobytes. The change landed in Python 3.14. The project's own issue explains the old value. The maintainers chose eight kilobytes in 2008, for a machine that ran a thirty-two bit operating system with under two gigabytes of memory.

One caution on that multiplier. The issue author measured it himself, on Python 3.11, writing a file he had downloaded over HTTP [S7]. A core developer confirmed the direction in the same thread, and posted no numbers [S7]. So nobody reproduced the figure [inference]. I quote it as the maintainers' stated reason for the change, not as a figure you can rely on.

Think what that does to your homework. Say some of you run 3.13 and others run 3.14. Your crossing counts then differ by a factor of sixteen. That reason has nothing to do with what the assignment measures. So report the version and the buffer size beside every number. One line of Python prints it.

12 12. Where does the difference come from?

On screen:

Part II

One answer, stated immediately, and then the instrument that proves it.

Twelve minutes. This part explains the number you just watched, and it introduces the instrument the rest of the course depends on.

I will tell you the answer immediately rather than build up to it. The answer is short and the interesting work is in the consequences.

First one elimination. It is not the disk. One mebibyte never reached a disk in either run. It is not the interpreter either, because both runs share it. So whatever the difference is, it happens between your loop and the operating system.

13 13. Where does the difference come from?

On screen:

Part II

The difference is the number of times each program crosses into the operating system.

*Your theory course defined a system call. This lecture counts them: each one makes a **crossing** out of your program, "a privilege transition, not a jump" [V15].*

A

Unbuffered

One call per byte, so 1,048,576 calls [inference].

B

Buffered

One mebibyte divided by the buffer size [inference].

=

So measure first

At 8 KiB, 128 calls. At 128 KiB, 8 calls [inference].

Here is the answer. The two programs differ in the number of crossings they make into the operating system [inference].

Program A asks once per byte. A mebibyte is one million and forty-eight thousand five hundred and seventy-six bytes, so that many requests.

Program B asks once per buffer fill. Divide one mebibyte by the buffer this machine allocated. At eight kilobytes that is one hundred and twenty-eight requests. At one hundred and twenty-eight kilobytes it is eight. Print the size before you divide, because I am not going to guess it for you.

Be precise about the vocabulary now, because I use it for the rest of the course. You already know what a system call is, so I am not going to define it. I am going to name the thing I count. The passage out of your program and back is one crossing, and every crossing is one system call. Do not call it a kernel call. Do not call it an API call. An API call may or may not become one, and that distinction is what this lecture exists to draw.

14 14. What one crossing costs

On screen:

Part II

4,972

instructions, one stat call

13,585

cycles for the same call

2,559

L3 lines evicted — your data, not the kernel's

Soares and Stumm measured this on 2010 hardware [S16]. The counts are not current [V2]. What carries over is the regime distinction, not the numbers [inference]. That distinction is a call rate. The lost cache state dominates at one call per 2,000 to 100,000 instructions. The transition cost takes over above that rate [S16].

[S16] FlexSC — Soares and Stumm, OSDI 2010. https://www.usenix.org/legacy/event/os-di10/tech/full_papers/Soares.pdf

Why is a crossing expensive? Your theory course probably said the processor changes privilege level. That is true, and it is not the main cost [S16].

Soares and Stumm measured what one system call does to the processor. A single stat call retires just under five thousand instructions in about thirteen and a half thousand cycles. Retiring an instruction means finishing it. The ratio is 0.37 instructions per cycle, so the processor finishes one instruction every three cycles [V2]. Straight-line code on the same processor finishes more than one per cycle, so this is slow work rather than ordinary work [speculative].

Now the third number. That one call evicted two and a half thousand lines from the level three cache [V2]. That is the processor's own cache, the last one before main memory. Those lines held your program's data. Each one becomes a cache miss that your code pays for after the call returns [inference].

Two caveats, and they are the kind this course exists to teach. The measurement is from 2010 hardware, so do not quote the counts as current. And the paper separates two regimes by call rate. The medium band is one call per two thousand to one hundred thousand instructions. In that band the lost cache state dominates. Above that rate the transition itself takes over. Both regimes are in the same paper [S16].

Program A makes one system call for every loop iteration. That loop probably runs fewer than two thousand instructions between calls, so it sits above the medium band [speculative]. That places it where the transition cost dominates [speculative]. The paper measures no Python program, and I have not counted the interpreter's instructions per call, so the placement is mine [speculative].

Either way the cost is largely fixed per call. Doubling the bytes in one call is nearly free. Doubling the calls is not.

15 15. What one trace line says

On screen:

Part II

A trace records the system calls a program made, in order, with arguments and return values.

[code]

1

The name

Which system call ran [S1].

2

The arguments

In parentheses, decoded [S1].

3

The return value

After the equals sign [S1].

A trace is not a log. A log carries what the program's author chose to print. A trace carries what the program did [inference].

[S1] strace(1) — Linux man-pages, which documents this example line. <https://man7.org/linux/man-pages/man1/strace.1.html>

This is one line of output from a tool called strace. It is the example the manual page itself uses, so it is a safe one to learn on [S1].

Read it left to right. The name of the system call. Then the arguments in parentheses, decoded into names rather than raw numbers. Then an equals sign and the return value.

That is the whole format. Every line looks like this. Once you can read one line you can read a thousand, and the rest of this course assumes you can.

One word about vocabulary. A trace is not a log. A log contains what the author decided to print. A trace contains every call the program made, including the ones the author never printed. So a tracer answers questions the author did not anticipate.

Read that line again, slowly.

16 16. Count the crossings

On screen:

Part II

The same two programs, run again under a counting tracer.

Command

strace -c -e trace=write python3 a.py [S1]

Program A

One write per byte [inference].

Program B

One write per buffer fill [inference].

First

Print io.DEFAULT_BUFFER_SIZE, then divide [S6].

Two runs

A traced run cannot be timed [V17].

[S1] strace(1) — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>

I am running the counting version now. The filter matters. Before your code starts, Python itself opens and reads a large number of files. Without the filter you get every call that work makes, and the one interesting line disappears into the noise [S1].

Look at the two summaries side by side. First I print the buffer size this machine uses. Then I divide one mebibyte by it. Then I compare my result with the count the tool reported. If those two disagree, the arithmetic used the wrong divisor. Finding that out is the exercise working rather than failing.

This is what I mean when I say the boundary is observable. You run one command and you get the exact number of requests your program made. Not an estimate, and not a model. Program A was slow because it made a million of them.

17 17. The count ratio over-predicts the time

On screen:

Part II

Illustration, not a measurement from this room: a count ratio of 100,000 beside a time ratio of 40 is not a mistake [inference].

time $\approx$ crossings $\times$ fixed cost + bytes $\times$ marginal cost [V1]

1

Two terms

Not a function of bytes alone [V1].

2

One term is shared

Both programs write the same mebibyte through the same loop [inference].

3

So counts over-predict

Under this model the count ratio exceeds the time ratio [inference].

This slide exists because of a sentence I was tempted to say and will not. The tempting sentence is that the ratio of the counts is the ratio of the times. It is false, and the run you just watched shows it false [V1].

Now apply it. Both programs write one mebibyte through the same million-iteration loop. So the second term, and all the interpreter work, is identical. It is a floor neither program can get under.

Under that model the count ratio over-predicts the time ratio, and the shared floor is why. If your counts differ by a hundred thousand and your times by forty, you have not made a mistake. You measured the floor. Those two figures are an illustration, not a measurement from this room.

One caveat on the model itself. Nobody has verified it across machines, and per-crossing costs vary by an order of magnitude between them [V1]. So treat it as a shape, not as a formula to solve.

You will see this gap in assignment one. Meet it here rather than there.

18 18. What a buffer actually does

On screen:

Part II

A buffer collects adjacent bytes and hands them down in one call.

[code]

Cost has two terms: the number of crossings, and the number of bytes [V1]. A buffer cuts the first and leaves the second alone, so the saving sits entirely in the call count [inference].

Look at the picture. Your loop still runs a million times. Every one of those writes still happens, and every byte is still copied [inference].

What changed is the right-hand arrow. Instead of a million crossings there are a few hundred, because the buffer holds bytes until it is full and then hands over a block.

I want to be precise about the word adjacent, because the next slide turns on it. A buffer holds bytes destined for consecutive positions in the file. It can do that because a block write to the operating system carries one offset and one length. Bytes going to one place travel together. Bytes going to two places cannot.

Say it back to yourselves in one sentence. A buffer saves calls, not work. The copying still happens. The loop still runs. Only the number of crossings goes down.

That is takeaway one, half stated. The other half is next, and it is the half the assignment measures.

19 19. Why a seek gives the saving back

On screen:

Part II

A buffer holds bytes for consecutive positions. Move elsewhere and those bytes cannot stay in it.

1

Constraint

Buffered bytes belong at the old offset [inference].

2

Consequence

Every seek forces the buffer out [inference].

3

Result

The buffered run converges on the unbuffered [inference].

The rule is buffering is fast when writes are adjacent, and buys little when they are not [inference]. Lampson gave the batching half in 1983 [S12].

If you keep one slide from today, keep this one. Assignment one is graded on the rule it states [speculative].

A buffer collects bytes for consecutive positions. Now suppose you write one byte, move to a different offset, and write another. The first byte belongs at the old offset. It cannot sit in the buffer waiting, because the buffer has one destination. So the library must hand it over before it can move.

Every seek forces the buffer out. Seek between every write and the buffer never holds more than one byte. The buffered program then makes the same number of crossings as the unbuffered one. The saving is gone, and the trace shows it.

So the rule has a condition. Buffering is fast when writes are adjacent, and buys little when they are not. Lampson wrote the batching half in 1983. The part about a seek emptying a userspace buffer is mine. It follows from the mechanism above rather than from him.

One honest limit. Adjacency is necessary and not sufficient. A program already writing a mebibyte in one call has nothing left to collect.

20 20. Does the seek itself cost a crossing?

On screen:

Part II

Yes. And a pair of system calls exists to remove it.

Naïve

lseek then write — two crossings for one operation [inference].

Merged

pread and pwrite "combine[s] file access and seek operations into a single command" [V3].

Measured

The merged pair cut the call count [V3]. For the same data volume the study also reports "a relatively reduced load in terms of overall duration" [V27].

Somebody always asks this, so I answer it before you do [speculative].

Yes. Moving to a new offset is itself a request to the operating system. So a random-access run does not make one crossing per byte. It makes two.

That is worth knowing for your assignment, because the random pattern is worse than the sequential unbuffered pattern rather than merely equal to it.

And here is the evidence that the crossing count is the real variable. Linux offers a pair of calls that merge the move and the access into one call. A study replaced the two-call pattern with the merged pair. The call count fell [V3]. For the same data volume the study reports a reduced overall duration [V27].

I am not claiming that the crossing count was why those calls were added. I do not know that, and the source does not say it. My claim is narrower. Merging two crossings into one gave a relatively reduced overall duration for that workload [V27].

You will not need these calls for assignment one. I mention them as evidence.

21 21. Trust the counts. Do not trust the times.

On screen:

Part II

The tracer stops your program twice per call. That destroys the timing and leaves the counting intact.

Every call stops

[V13]

So "the counts are trustworthy and the absolute times are not".

Counts only

Gregg's dd loop

[V17]

One byte per call, "runs 442× slower under strace".

His machine

Re-runs

[V31]

"roughly 37× to 136×", other hardware.

Not a bound

The range does not contain Gregg's figure. Each number measures one machine and one workload [inference].

You have no reason yet to believe this tracer, so let me give you one, and let me tell you where it misleads [speculative].

Because it stops every call that reaches the kernel, it counts every one of them. Nothing is sampled. That is why you may believe the counts.

There is one gap worth knowing. Linux serves a few calls without a crossing at all, from a page it shares with your process. The tracer is silent on those, because there is nothing to intercept [V26].

Because it stops every call, it also makes your program much slower. Brendan Gregg measured the worst case with a dd loop that copies one byte per call [V17]. That program does almost nothing but system calls, and it ran four hundred and forty-two times slower under the tracer [V17]. Later runs of the same loop report thirty-seven to one hundred and thirty-six times [V31]. Those runs used other hardware, with other speculative-execution mitigations [V31]. Notice that the later range does not contain

Gregg's figure. None of the numbers is wrong, because the overhead follows the call rate of one program on one machine [inference]. So a single overhead number is not a fact about the tool [inference].

So the rule for this course is: count under the tracer, time without it. That is why part one and part two used two different runs, and I said so at the time.

If you benchmark under a tracer in the lab, you will report nonsense.

22 22. Windows is absent here, on purpose

On screen:

Part II

I used one Linux tool here and no Windows tool. Here is the reason, before you infer one.

1

Why

The Windows tracer has no counting summary to match [speculative].

2

Where it goes

Section five, with a real Windows trace [S10].

3

The promise

Equal footing is a course promise, not a per-lecture one.

[S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/>

Some of you run Windows. You have just watched twenty minutes of Linux and I do not want you drawing the obvious conclusion [speculative].

The reason this section is Linux-only is narrow and technical. The counting summary I used is a feature of this particular tool. The Windows tracer this course uses is a different design and offers no equivalent one-line summary. Counting was the job here, so I used the tool that counts.

The course position is this. Neither platform is the reference implementation in PB158. The Windows toolchain gets its own depth in a later lecture, and the labs exist for both.

23 23. It is not one buffer — it is a stack

On screen:

Part III

We counted the crossings. Now we ask where the bytes actually are.

Fifteen minutes, and this is the structural part of the lecture.

Section two left you with a good model and one obvious question. If the library holds my bytes and hands them down, where are they afterwards? The answer is not "in a buffer". It is "in one of several, and you cannot tell which without looking".

I draw one diagram in this section. I refer back to it in the durability lecture, in the memory mapping lecture and in the crash analysis lecture. If you photograph one slide today, photograph the next one.

24 24. It is not one buffer — it is a stack

On screen:

Part III

[code]

Levels 1 and 2 are alternative routes into level 3 [V6]. Each arrow is one call [S17] [S3] [S13] [V37] [V33].

[S17] `io.IOBase.flush()` — Python documentation. [S3] `fflush(3)` — Linux man-pages.

[S13] `fsync` — The Open Group.

Four levels. Read them from the top [inference].

Level one is your program's own buffer, owned by the language runtime. Level two is the C library's buffer. Level three is the operating system's own memory for file data. Level four is the cache inside the storage device. Then, at the bottom, the medium.

Notice that levels one and two sit side by side. They are alternative routes, not a pile. A Python program uses the buffer at level one and calls the kernel directly. A C program uses the buffer at level two. I come back to this in four slides, because it is the detail people get wrong [V6].

Now read the arrows. Each one is a different call, and each one moves your bytes down exactly one step. No single call takes data from the top to the bottom.

Look at the bottom arrow. I come back to it, and I am not going to skip it.

25 25. Four levels, and who empties each one

On screen:

Part III

LEVEL	WHO OWNS IT	EMPTIED BY
1 Application	the runtime's own buffer [S8]	<code>f.flush()</code> [S17]
2 C library	the <code>stdio</code> buffer [S3]	<code>fflush()</code> [S3]
3 Kernel	the page cache [V32]	<code>fsync()</code> [S13]
4 Device	the drive's write cache [V20]	a cache-flush command [V33]

The page cache lives in the machine's memory [V32]. It is not the disk's cache — the disk has its own, one level further down [V20].

Reads use the same four levels, in the other direction. The kernel fetches pages ahead of a sequential reader, so "a sequential `read()` just happens to return faster because the data was already fetched" [V34].

The same four levels as a table, because you will want to copy this one [speculative].

Level three deserves its own name. It is called the page cache, and it is the operating system's own buffer for file data. It lives in the machine's memory. A write that reaches it has left your program, and it has not left the machine.

Two things people get wrong, and you should get neither wrong.

First, it is not the disk's cache. The disk has its own cache, and that is level four, one step further down. Two caches, two owners, two failure modes.

Second, a busy machine shows very little free memory because the page cache has grown into it. That is correct behaviour and not a leak. The memory is reclaimable the moment something else needs it.

One sentence about reading, because this hour only ever writes. The same four levels carry data upward. Level three fetches pages ahead of a sequential reader, so the read returns from memory rather than from the device [V34]. This stack is not a write-only structure, even though every demonstration today is a write [inference].

Now look at the last row. The device has a cache of its own, and emptying that one is the least reliable step in the diagram. Hold that thought.

26 26. To flush is to cross one boundary

On screen:

Part III

The verb means one thing here: to hand a buffer's contents to the next level down.

The C library manual says it plainly: "Note that fflush() flushes only the user-space buffers provided by the C library. To ensure that the data is physically stored on disk the kernel buffers must be flushed too" [S3].

✗

Do not say

Save, commit, persist — all claim durability [inference].

✓

Say

Flush. It names one boundary [inference].

[S3] fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

This is the misconception I most want to remove today, and it earns a slide of its own [speculative].

The common belief is that the verb flush means "written to disk". It does not. It means "handed to the next level down". The manual page says so in its own words. It carries that note because the question comes up so often.

Read the quote carefully. It flushes only the user-space buffers provided by the C library. To get data physically onto a disk, the kernel buffers must be flushed too. That is two calls, not one.

So in this course, flush is a verb with a precise meaning. It crosses one boundary. If you write "save" or "persist" in your assignment when you mean flush, I will ask which boundary you crossed.

A student who leaves believing that a program flushes all the way to the platter has learned something false. Ordinary use will not correct it, because only a power cut falsifies the belief [inference].

27 27. Wholesale down, retail up

On screen:

Part III

Every level buys a large block from the level below. It sells small pieces to the level above.

That asymmetry is why every level in this diagram holds a buffer [inference]. Each level's "unit of purchase is larger than its unit of sale" [V18].

1

Buys in blocks

4096 bytes on an ext4 file [V7].

2

Sells in bytes

Your loop writes one byte [inference].

3

Same shape elsewhere

Allocators, caches, network stacks [speculative].

Four levels is four facts, and four facts are hard to keep. One rule is easier, so here is the rule that generates all four [V18].

Each level buys wholesale from the level below and sells retail to the level above. Its unit of purchase is larger than its unit of sale. That asymmetry is why every level in the diagram holds a buffer.

Apply it to level two. On an ext4 file the C library takes a buffer of four thousand and ninety-six bytes [V7]. It sells single bytes to your loop. The gap between those two units is the saving you measured in part one.

I give you the rule rather than the four facts because the rule travels. You will meet it in memory allocation, in processor caches and in network buffers.

I avoid one thing deliberately. I am not giving you a warehouse or a bucket for the page cache. Physical memory is a cache rather than a container, and a container image installs the wrong intuition.

28 28. Python's buffer does not sit on the C library's

On screen:

Part III

Levels one and two are alternatives, not a pile.

Intuitive, and wrong

A Python write falls through the runtime buffer into the C library buffer and then into the operating system [inference].

What happens

"CPython reimplements the same policy above libc rather than inheriting it" [V6].

Python uses level one. C uses level two.

Python documents a typical 128 KiB [S6]. The C library takes 4096 on an ext4 file [V7].

Two adjacent levels, two sizes, neither in your source [inference].

[S6] Built-in Functions, open() — Python documentation. <https://docs.python.org/3/library/functions.html#open>

I flagged this on the diagram and I am returning to it, because it is the detail people get wrong [speculative].

If you read levels one and two as a stack, you conclude that a Python write passes through both. It does not. CPython reimplements the buffering policy above the C library instead of inheriting it. A Python program uses level one. A C program uses level two. They are alternatives.

Here is evidence you can check yourself, and it is the best evidence today that the stack is real. Python documents a typical buffer of one hundred and twenty-eight kilobytes [S6]. Now take an ext4 file with a block size of four thousand and ninety-six bytes. On that file the C library buffer measures four thousand and ninety-six bytes too [V7]. That is one measured machine. Do not carry the number to a machine you have not measured.

Two adjacent levels. Two sizes, a factor of thirty-two apart. Neither number appears in your source code.

29 29. The bottom level acknowledges early too

On screen:

Part III

Level four is the uncomfortable one, and skipping it would make the diagram dishonest.

1

The device holds a buffer

"most HDDs and many SSDs acknowledge a write once it lands in an onboard DRAM cache" [V20].

2

The standard allows a no-op

On fsync: "a null implementation is permitted" [S13].

3

Software cannot check

No trace separates a device that flushed from one that only reported it [V5].

[S13] fsync — The Open Group. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>

I could stop the diagram at three levels. I am not going to, because the fourth is where data is actually lost, and no tool in this lecture can watch it [V20] [V5].

The device has its own memory. Most spinning disks and many solid state drives report a write as complete when it lands in that memory. At that moment the bytes are still short of the platter or the flash. A power cut in that window loses the data. The device already told the operating system that the write succeeded.

And here is the limit on today's thesis, which I would rather state than have you find. A trace cannot separate a device that flushed from a device that only reported that it flushed. Everything above this level is observable with a tool you can run. This level is not.

That is why a durability lecture exists, why databases are written the way they are, and why people test storage with a power switch.

30 30. Three lines that print nothing

On screen:

Part III

A demonstration. You have the stack now, so predict this one.

[code]

To a terminal

Does hello appear?

Redirected to a file

What does the file contain?

Write down both answers before I run either one [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Three lines. Import, print, exit [inference]. Nothing unusual anywhere.

Same method as part one: predict first, out loud. But this time I expect most of you to be right, and that is why this sits here rather than at the beginning.

Two runs. First in a terminal. Then redirected into a file, and I show you the file.

Take thirty seconds. Write two answers. Does hello appear on the terminal, and what is in the file?

Hands up if you think the terminal shows hello. Hands up if you think the file contains hello.

Now watch. Terminal first. And now redirected.

The file is empty. Zero bytes. If you predicted that, you did not guess. You used the diagram from the start of this part, and that is exactly what this demonstration tests.

31 31. Why the file is empty

On screen:

Part III

The rule sits in the C standard, not in the kernel.

1

Mode follows destination

Output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5].

2

A terminal is interactive

Bytes go out per line [inference].

3

A file is not

Bytes wait [inference].

4

The exit skips cleanup

No write reaches the kernel [inference].

[S5] C99 draft §7.19.3. <https://busybox.net/~landley/c99-draft.html>

Here is the explanation, and notice where it comes from [S5].

The rule is in the C language standard. It is not an operating system behaviour and it is not a Python quirk. The standard says that standard output is fully buffered if and only if the stream can be determined not to refer to an interactive device.

Then the third line ends the process without running any cleanup. The standard explicitly permits termination paths that do not close files properly. The buffer is process memory. The process is gone. The bytes went with it.

Now the sentence that connects this back to section two. A trace of this program holds no write call at all. Not a failed one. Not a short one. None. The bytes never got as far as the operating system.

That is the top of the stack, proved to exist by removing it.

32 32. What this demonstration does not prove

On screen:

Part III

It proves level one is real. It proves nothing about levels three and four.

"the honest classroom demo is a power-cut, not a kill -9 . Killing the process loses only the runtime's buffer; the kernel still writes back." [V14]

1

What died

The process, and the level-one buffer with it [inference].

2

What survived

Anything already in the page cache [V14].

3

Wrong conclusion

"So flush() was enough." It was enough against this test [inference].

I have to bound that demonstration immediately, because it is easy to over-read [speculative].

Killing a process destroys that process's memory. That is level one. The program gave some bytes to the operating system earlier. Those bytes sit in the page cache. The operating system writes them back in its own time. Your process does not need to exist.

So this proves that level one exists and can lose your data. It proves nothing about levels three and four.

The honest test for the lower levels is a power cut, not a kill. Pull the plug and you find out what the page cache was holding. Nobody is doing that in this room today.

Why so careful? Because students test with a kill, watch the file survive, and conclude that the program flushed everything it needed to. That conclusion holds against a kill. It does not hold against a power failure.

Same demonstration, correct conclusion. The difference is which level you claim it tested.

33 33. Which levels can you actually watch?

On screen:

Part III

I promised three boundaries of four. Here is the tool each one needs.

App → library

[V10]

"printf is visible to ltrace and write to strace".

Another tool

Library → kernel

[S1]

Where our tracer attaches.

Today

Kernel → device

[V30]

Block-layer tracing, not in this course.

Elsewhere

Device → medium

[V5]

No trace separates a flush from a report of one.

Never

On the first slide I said three of the four boundaries are observable. This is the slide where I show you which three, and why the fourth is not one of them [V5].

There are four boundaries in the diagram. Here is what each costs to see.

The top one, your program calling into a library, needs a different tool. It is called ltrace. It attaches to library calls instead of system calls, so it shows printf where our tool shows write. Run the two tracers on one program and their outputs disagree. That is how you test takeaway two rather than believe it, and it is a lab exercise rather than something you do today.

The second boundary is the one we have watched all hour.

The fourth is not observable from software at all.

So: three of these four you can watch, one of them with a tool you now know. The one you cannot watch is where data actually gets lost. That asymmetry is section six.

34 34. The same write, at three levels

On screen:

Part IV

Now that the levels have a reason to exist, walk down them with one small program.

Eight minutes. Section three said the levels exist and why. This section walks down them with a program you can compile yourself.

You write C in this programme, so I am not going to apologise for the C or detour through pointers. We read the core of it. I compile it in front of you. I run it under the tracer. Then we read its trace.

35 35. The same write, at three levels

On screen:

Part IV

LEVEL	PYTHON	C	BUFFERED?
Application	<code>f.write(b"x")</code> [S6]	—	yes [S6]
C library	—	<code>fwrite(...)</code> [S3]	yes [S3]
System call	<code>os.write(...)</code> [S20]	<code>write(...)</code> [S2]	no [inference]

The bottom row takes a **file descriptor** — the small integer naming an open file. You saw one already: the `= 3` in the trace line was a file descriptor [S1].

[S1] *strace(1)* — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S2] *write(2)* — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S3] *fflush(3)* — Linux man-pages. [S6] Python documentation, *open()*. [S20] *os* — Miscellaneous operating system interfaces, *os.write()*. <https://docs.python.org/3/library/os.html#os.write>

Read the table by column first, then by row [speculative].

The Python column and the C column are two routes to the same bottom row. Python's file objects hold a buffer at level one. C's *fwrite* fills a buffer at level two. Both call *write* in the end, and *write* is the system call.

Now read the last column, because it carries the point. Each level holds a buffer, or it passes the bytes straight down. A Python file object holds one. The *os.write* function does not, because it wraps the system call directly. In C, *fwrite* fills a buffer and *write* does not. The levels are not decoration. They are the mechanism you measured in part one.

The bottom row introduces a term I deferred until now. A file descriptor is the small integer a Linux program uses to name an open file in system calls. You have seen one: the *open* call returned three. It will appear as the first argument of every *write* to that file.

Remember the word. In section five it is the first thing that differs.

36 36. Meet the program: `iodemo.c`

On screen:

Part IV

Nine lines of C. The first open fails on purpose, so the trace shows an error path.

[code]

It prints nothing, on purpose. A silent program makes a legible trace [speculative].

Nine lines on the screen. The full file adds comments and error returns.

Step one opens a file that does not exist, and that failure is deliberate. If every call succeeds, the trace is success all the way down, and the most interesting difference between the two platforms never appears. I show you that difference in section five.

Step two opens the file we mean to write.

Step three writes fourteen bytes in three calls. The C library collects them in its own user-space buffer before any of them crosses into the kernel [S3] [S5]. Watch that number.

Step four closes the file and checks the return value of the close. Almost none of the C I have read does that [speculative]. I come back to why in section six.

The program prints nothing at all. My reason is the plain one. A silent program makes a legible trace.

I am compiling it now, with warnings on, and running it.

37 37. Three calls in the source. One in the trace.

On screen:

Part IV

3

fwrite calls in the source

14

bytes written in total

1

write call in the trace

The three fwrite calls go into one C library buffer. The manual calls that level "the user-space buffers provided by the C library" [S3]. The line to look for in the live trace, predicted from the source [inference]:

[code]

The 3 is the file descriptor. The 14 is all three fwrite calls, in one crossing [inference].

[S3] fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

This is where takeaway two stops being a claim and becomes something on a screen [inference].

Three fwrite calls in the source. Fourteen bytes. One write in the trace.

Point at the source. Point at the trace. There is no line-by-line correspondence between them, and there never was. What you write is a library call. What the operating system sees is a system call. Your source did not say how many system calls to make. The buffering policy in the library fixed that number.

The line on the slide is a prediction. I wrote it from the source before I ran anything. Now watch the real trace. Check it against the prediction. If they differ, the prediction was wrong and the machine was not.

And notice when the write happens. Not at the third `fwrite`. It happens when `fclose` runs, because `fclose` hands the buffer down to the kernel. The call in your source and the call in the trace are not even at the same point in time.

38 38. `fwrite` takes a size and a count

On screen:

Part IV

Four arguments, and two of them multiply.

[code]

1

It returns a count

Items written, not bytes. The count equals bytes only when the item size is 1 [S2].

2

Fewer is legal

Below it, "a successful write() may transfer fewer than count bytes" [S2].

[S2] write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S21] fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>

A small thing that trips people. It costs thirty seconds now or twenty minutes in the lab [speculative].

The call takes four arguments. A pointer to the data, the size of one item in bytes, the number of items, and the stream. In our program I passed one and six, meaning six items of one byte each. I could have passed six and one and written the same bytes.

Why does it matter? Two reasons.

First, the return value counts items, not bytes. The manual says the count equals the number of bytes only when the item size is one [S21]. If you write six items of one byte and the call returns four, you wrote four bytes. If you write one item of six bytes and it returns zero, you wrote nothing.

Second, fewer is legal. The system call underneath may transfer fewer bytes than you asked for, and the manual says so. The library call has its own version of that: it may report fewer items than you asked for. Two different contracts, one habit. Read the return value.

39 39. Two operating systems, side by side

On screen:

Part V

Read as a comparison. Running the tools is the first lab's job.

Twelve minutes, and some of you waited for this section.

A warning about what this is not. It is not a demonstration that one platform is better. It is not a tutorial on installing a Windows tracer. It is a reading exercise on two pieces of output. The lesson is a structural difference between the systems, not a difference in formatting.

I also need to be careful about what evidence I have. I will tell you exactly what I am showing you, where it came from, and what it cannot do.

40 40. Why NtTrace and not Process Monitor

On screen:

Part V

Two Windows tools, two layers. One answers this lecture's question.

Process Monitor

1

Shows "real-time file system, Registry and process/thread activity" [S18]

2

The usual Windows choice [speculative]

3

The wrong layer for us [inference]

NtTrace

4

Reports native API calls, in user mode [S10]

5

Runs on the debugger interface, so it "needs no driver and no administrator rights" [V21]

[S18] Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

Process Monitor is the tool people usually reach for on Windows [speculative]. Microsoft describes it as showing file system, registry and process activity [S18]. Those are filesystem events rather than system calls, so it watches the wrong layer here [inference].

So we use NtTrace. It watches the native API, which is the last user-mode layer before the kernel.

Be precise here, because I could overstate it. This is not the same attachment point as the Linux tracer. The Linux tracer sits at a documented kernel boundary. NtTrace sits one level above the kernel, in a user-mode library. That gap is itself this section's lesson, and I will come back to why the gap exists.

One practical point matters more than it sounds. NtTrace needs no administrator rights, because it is a debugger rather than a driver. You can run it on a lab machine without changing that machine's security settings. The alternative on Windows is a kernel-mode tracer. That tool needs a signed driver and a boot-time change. Nobody can install one of those in a teaching lab.

I will also be honest about the trade. Neither of today's tools is what people run in production, and I come back to that at the end.

41 41. Three Windows words you will need

On screen:

*Part V**Win32*

The documented interface a Windows program calls. The 32 is history, not a width [inference]. Microsoft renamed it the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22].

The native API

The layer below Win32, inside ntdll.dll. Microsoft calls it "internal to the operating system" [S9].

The object manager

The kernel component that names open objects. Its paths begin ??\ [V22].

[S9] `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation> [S22] Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>

Three words, thirty seconds, and then the Windows trace makes sense [speculative].

Win32 is the documented interface a Windows program calls. It is the Windows equivalent of the C library plus the POSIX calls. It is the interface the documentation tells you to use. The name is historical. Microsoft renamed the same thing the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22]. So the 32 in the older name tells you nothing about the width of a program or of a trace [inference]. The trace you see next is genuinely a thirty-two bit one, and the two facts are unrelated [inference].

The native API is the layer below Win32. It lives in a library called ntdll, and it is the last user-mode code before the kernel. Microsoft's own documentation calls it internal to the operating system.

The object manager is the kernel component that names open things — files, processes, events. Its paths look different from the paths you type, and one of those paths is on the next slide.

42 42. A real Windows trace

On screen:

Part V

You can already read this. The anatomy matches the Linux line, and the vocabulary differs [inference].

[code]

Three lines excerpted from Roger Orr's published trace of `cmd` [S10]. Ignore every argument except the handle, the path and the return value [inference].

[S10] NtTrace — Roger Orr, project page. <https://rogerorr.github.io/NtTrace/>

This is real Windows trace output. It is the worked sample published on the tool's own project page, by the person who wrote the tool [S10].

Notice first that you can already read it. Name, arguments in parentheses, return value after the arrow. Same anatomy as the one Linux line I made you read, which is exactly why I made you read that line.

Ignore almost all of it. I am asking you to look at exactly three things: the handle, the path string, and the return value. Everything else on this slide is there because I am not allowed to edit somebody else's trace.

Now what this trace is not, because I would rather you heard it from me. It is a trace of the Windows command shell, not of our program. It is old — the addresses are thirty-two bit, from a much earlier Windows.

So it establishes what a Windows trace looks like. It does not establish the comparison. I have three things to show you in it, and all three are structural.

43 43. A handle is not a file descriptor

On screen:

Part V

First difference, visible in the first argument of almost every line.

Linux

A file descriptor. A small integer indexing a per-process table. The trace showed = 3 [S1].

Windows

A handle. An opaque value naming an open object. The trace shows FileHandle=4 [S10].

Same job in a trace, different mechanism in the system. Do not use one word for the other [inference].

[S1] strace(1) — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/>

First difference, and it sits in the first argument of nearly every line on both systems [S10].

On Linux you get a file descriptor. A small integer indexing a table that belongs to your process. Three, four, five. You can reason about the numbers.

On Windows you get a handle. An opaque value that names an open object, and the object need not be a file. It might be a process, an event or a registry key.

In a trace they occupy the same position and do the same job, which is why it is tempting to use one word for both. Do not. In this course I hold the two words apart. If you write handle in a Linux assignment, I will ask which system you meant.

This is a small thing. I spend a slide on it because it is the cheapest instance of today's platform lesson: the same job, two mechanisms, visible in one argument.

44 44. Two error namespaces in one return value

On screen:

Part V

Second difference, and it is the layer boundary showing up where nobody expects it.

[code]

1

First number

An NTSTATUS, the native layer's code [S10].

2

Second number

The Win32 error it translates into [S10].

3

Linux

One errno. Nothing to translate [inference].

[S10] NtTrace — Roger Orr, published sample trace. <https://rogerorr.github.io/NtTrace/>

Second difference, and this is the one I find genuinely interesting [speculative].

Look at that return value. It carries two error codes at once. The first is the native layer's own code. The second is the Win32 error the first translates into, with the message text you would actually see.

Two namespaces, in one return value, from one call.

Linux has one. A system call fails and you get a number, and that number is the error. There is nothing to translate, because only one layer names errors.

Why does Windows have two? Because there genuinely are two layers there, and both are real interfaces with their own error vocabularies. You are looking at the seam between them, visible because the tracer sits below the translation and reports both sides.

This is why the demo program opens a file that does not exist. Without a failure this line never appears, and a clean run hides the seam between the two layers.

45 45. The stable boundary sits at a different level

On screen:

Part V

Third difference, and the other two are symptoms of it.

The Windows path reads "\\??\\C:\\WINDOWS\\system32\\cmd.exe.Local". That prefix "is the object-manager namespace, which a Win32 program never types" [V22].

Linux

[V8]

The system call interface is the public contract.

Syscall

Windows

[S9]

Native entry points are "subject to change from one release of Windows to another".

Win32

[S9] *winternl.h* — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

Third difference, and the other two are symptoms of this one [inference].

Look at the path in the Windows trace. It starts with a backslash, two question marks, a backslash. No Win32 program ever types that string. It is the name of the file in the object manager's namespace. That namespace sits below the interface a Windows programmer uses. The tracer is showing you a layer programs do not normally see.

Microsoft says so in its own documentation. The native entry points are internal to the operating system and subject to change between releases of Windows. That is a first-party statement.

On Linux the opposite holds. The system call interface is the public contract. You may write a system call number down and rely on it.

Now the careful sentence, and it is the takeaway. Both systems are stable at the layer they nominate. The disagreement is about which layer gets nominated. Everything else follows from that single placement, including where a tracer can attach.

46 46. Where the boundary sits is contested history

On screen:

Part V

Two readings survive the evidence. I will not pick one for you.

1

Reading A: deliberate

POSIX, OS/2 and Win32 each reached NT through the native API [S19].

2

Reading B: vestige

A reported POSIX subsystem that was "never really completed" [S15].

3

Linux had no choice

It "had to make the kernel boundary itself the stable interface" [V38].

[S15] *The sad history of the Microsoft POSIX subsystem* — Reiter (2010). <https://brian-reiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/> [S19] *Inside Native Applications* — Russinovich (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>

I said the placement was a decision. Let me be more careful, because the evidence supports two readings and I would be overstating if I gave you only one [S15] [S19].

Reading A is design. NT had three operating environments: POSIX, OS/2 and Win32 [S19]. Each one served its own clients through its own interface. Each one reached NT through the native API [S19]. The source does not say where those environments ran, so neither do I. On that reading the split is deliberate. An unstable lower layer then follows from it [inference].

Reading B is vestige. A POSIX subsystem did exist. One practitioner account says it was built to satisfy the procurement rules of the United States government. It was never finished. On that reading the split is what remained after the plan was abandoned.

Both accounts are in the literature and I am not going to settle it in a practicum. The mechanism is settled. The intent is not.

Linux is the clearer case. It was a kernel built against a separately developed userland it did not control [V38]. It could not update the C library in lockstep, so it had to make the kernel boundary itself the stable one [V38].

47 47. Linux is the outlier, not Windows

On screen:

Part V

The tempting conclusion is backwards, and the evidence is a list of other systems.

"Linux is the only mainstream system that treats the raw syscall interface as a public contract." [V9]

1

Who declines the promise

FreeBSD, OpenBSD, NetBSD, illumos, macOS — and not equally [V36].

2

Why they can

Each ships its own C library [V9].

3

So say different

Never say worse [inference].

Now the correction to the conclusion you were probably forming [speculative].

After the last two slides it is tempting to think Windows is the odd one out. It is not. Look at the list: FreeBSD, OpenBSD, NetBSD, illumos, macOS. None of them publishes a Linux-style promise about the raw system call interface.

They are not equally strict about it, and I should say so. FreeBSD compiles in compatibility layers, so in practice it holds the ABI while declining to promise it formally [V36]. Read the list as "nobody else promises it in the same way", not as "everybody else breaks it".

Linux is the exception. It is the only mainstream system that treats the raw system call interface as a public contract.

In this course you say the platforms are different. You do not say one is worse. If you write that, I will ask which property you are ranking them on.

48 48. What that trace cannot do

On screen:

Part V

The honest caption, so nothing here is stronger than the evidence.

1

Not a matched pair

"It is not a matched pair." It follows the shell, not our program [V23].

2

Not what you will see

A 32-bit, XP-era trace, not today's addresses [V35].

3

One maintainer

And the layer it watches changes between releases [S9].

4

What survives

The handle, the path, the two error namespaces [inference].

[S9] `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

I promised to tell you what my evidence cannot do, so here it is on its own slide [speculative].

First, the two traces are not a matched pair. The Linux material and the Windows material do not come from the same program on the same day. A proper side-by-side needs both traces of one program, produced together, and that pair does not exist yet. I will produce it before the first lab and put it in the course repository.

Second, this is not what you will see. That trace is thirty-two bit and XP-era. A run on a lab machine today shows sixty-four bit addresses instead [V35].

Third, this tool has one maintainer, and Microsoft documents that layer as changing between releases. That is thin ice and you should know you are standing on it.

What survives all three caveats? Every claim I actually made. A handle rather than a file descriptor. An object path a Win32 program never types. Two error namespaces in one return value. Those are structural facts, and none depends on the year of the trace.

49 49. What we have not looked at

On screen:

Part VI

One question I will not answer, and the work that follows from it.

Six minutes left, and I am not going to spend them listing topics.

I will put one question back on the screen. I will not answer it. Then I show you two consequences of the stack. You already have everything you need to see them.

50 50. `write()` returned success. The bytes are not on a disk.

On screen:

Part VI

The whole hour built towards this sentence. I leave it unresolved.

The manual says it plainly: "A successful return from `write()` does not make any guarantee that data has been committed to disk" [S2].

Saltzer, Reed and Clark gave the general form in 1984. A lower level may offer "an incomplete version of the function... useful as a performance enhancement" [S11].

So when is it actually safe? That is the durability lecture's question, and this lecture leaves it open.

[S2] `write(2)` — Linux man-pages. [S11] End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984). <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

Read the first quote again. It comes from the manual page for the most basic write call there is. It says a successful return guarantees nothing about a disk [S2].

That should now be obvious rather than alarming, because you have the stack. A successful write means the bytes reached the next level down. Which level depends on which call you made.

The second quote is there so you do not think this is a bug someone will fix. Saltzer, Reed and Clark wrote it in 1984. A lower level may offer an incomplete version of a function as a performance improvement. The complete version needs the application at the end points. Every level in our diagram is an instance of that.

So: when is it actually safe? I am not answering today. I want you to leave with the question open, because a programmer who believes it is already answered writes no durability step at all.

51 51. Buffering moves your errors

On screen:

*Part VI**The error arrives somewhere other than where you caused it.**1**Why it moves**The bytes only reached a buffer [inference].**2**Where it arrives**Where the library flushes, so at fclose [inference].**3**The rule**Check the return value of the call that flushes [inference].**4**The idiom hides it**"the most common durability idiom in every mainstream language... contains no durability step anywhere" [V4].*

Here is the practical consequence, and it will cost you a debugging afternoon at some point [speculative].

Your fwrite returned success. It returned success because the bytes reached a buffer. A full disk, an unplugged device, an exceeded quota — none of that is known yet, and none of it can be.

That is why the demo program checks the return value of fclose. Almost none of the C I have read does [speculative]. That single check is the difference between having written fourteen bytes and believing you did.

The last row is worth saying slowly. The most common way to write a file in every mainstream language contains no durability step anywhere.

Python differs here. A Python with block does raise if the close fails. The two languages differ in what they force you to notice, which is the property I am ranking them on.

This is error reporting, not error recovery. Recovery belongs to the durability lecture.

52 52. Never retry a failed close

On screen:

Part VI

The obvious repair is wrong. Three storage engines dropped the retry in 2018 [V40].

"Retrying the close() after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4].

Three databases reached the same conclusion one level down, for fsync : PostgreSQL, MySQL's InnoDB and MongoDB's WiredTiger [V40]. WiredTiger wrote the reason into its own source comment: "we don't retry fsync-style calls and panic if a flush operation fails" [V24].

[S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

Now you check the return value of close. You will want to do the obvious thing with it, and the obvious thing is wrong [S4].

Do not retry. The manual page says why, and the reason is specific. After a failed close the descriptor may already be gone. If another thread has opened a file meanwhile it may have been given that same number. Your retry closes somebody else's file.

Think how strong that is. The three have three different architectures and arrived at the same answer within months [V41]. All three chose to crash the program deliberately when fsync fails [V40]. None of them tries again.

The transferable rule: when a failure leaves you unable to tell what state you are in, stop the program and report it. Continuing hides the damage and does not repair it.

53 53. Networking and IPC are this same mechanism

On screen:

Part VI

Not new subjects. The same objects and the same calls, wearing different names.

1

A socket on Linux

It is a file descriptor, and Linux makes that contract binding [V28].

2

A socket on Windows

An opaque SOCKET, and Windows makes Winsock the contract [V28].

3

A pipe

A file descriptor too, so the same calls reach it [V29].

Depth belongs to the next lecture. The job here is to stop you filing networking under "something else".

Thirty seconds, and then I stop naming things [speculative].

When you meet networking in this course it will not be a new mechanism. On Linux a socket is a file descriptor. The same read and write calls work on it. Sockets, pipes and terminals are all file descriptors, which is why one program can wait on all of them at once [V29].

On Windows a socket is an opaque type rather than a file descriptor. The binding contract is Winsock, the Windows sockets interface, rather than the file-descriptor contract [V28]. That is section five's pattern showing up again in a place you did not expect it [V8]. Same job, different mechanism.

54 54. Honest limits

On screen:

Part VI

What today's tools cannot see.

•

The device level

[V5]

Not observable from software: no trace can check it.

•

Asynchronous I/O

[V11]

The tracer sees the setup, then no I/O [inference].

Here is what today did not establish [speculative].

The device level is not observable from software. I said it in section three and I repeat it here. It is the limit on today's thesis, and it is where data is actually lost.

Asynchronous I/O breaks the model you have just learned, and it breaks it interestingly. A modern Linux interface moves submission into shared memory rings. One option starts a kernel thread. That thread polls the rings. Steady-state submission then makes no system calls at all. The tracer sees the setup and then nothing. The boundary you just learned to watch is already moving.

55 55. Honest limits, continued

On screen:

Part VI

And what today's tools cost.

•

strace in production

[V25]

Even the cheaper --seccomp-bpf mode [V39] "should therefore not be used on production systems!"

•

Mapping, crash dumps

—

Each has its own lecture.

And the tools. The tracer has a second and cheaper mode, which you select with the option `--seccomp-bpf`. On the workload of rebuilding strace itself, plain strace costs 3.06 times the untraced run and that mode costs 1.44 times [V39]. A strace contributor disqualifies it anyway. The mode "still stops at each syscall of interest and should therefore not be used on production systems!" [V25]. What people run in production is a dif-

ferent mechanism: eBPF on Linux and event tracing on Windows [speculative]. I have no per-call figure for either, so I am not giving you one. Today's tools watch the layer this lecture is about, which is why the course uses them.

Finally, memory mapping and crash dumps. Each has a lecture.

56 56. What to do on Monday

On screen:

Part VI

1

Assignment 1 measures takeaway 1

Two settings, two patterns, one mebibyte.

2

Start from section one

Add the random-access variant [inference].

3

Report your buffer size

Beside every number.

4

Do not benchmark under the tracer

Count traced, time untraced [V17].

5

Count in the dev container

Both platforms count there.

6

Week 2 opens

On 22 September.

The last thing I want you to hear is work rather than topics.

Assignment one measures takeaway one. Buffered against unbuffered, sequential against random, one mebibyte, and an explanation in Markdown. Four numbers and a paragraph.

The two programs from part one are your starting point. Add the random access variant and you have the whole experiment. You now have the model to write the explanation. I mark that part.

One thing for the Windows half of the room, because I owe you it after part two. The counting tool is Linux-only, and the assignment asks for counts. The course ships a dev container for this. It is a prepared Linux environment that VS Code starts for you. It holds the same tools on a Windows machine and on a Linux one. Lecture one told you to install it. Nobody is asked to produce a count their platform cannot produce.

Three rules from today that will earn you marks. Report your version and buffer size beside every number. The default moved from eight kilobytes [S7] to a typical one hundred and twenty-eight [S6]. Count under the tracer. Time without it. Say in your write-up that you did both. And when your count ratio and time ratio disagree by orders of magnitude, do not assume a mistake. Explain the gap. The explanation carries more marks than the numbers do.

57 57. Summary — what follows

On screen:

Closing

1

Ask "accepted or done?"

A fast return reports acceptance, not completion [V12].

2

Count, do not reason

No level reports which one holds your data [V16].

3

Name the level

Your buffer, the C library, the page cache, the device [inference].

4

Measure your own machine

Never quote a remembered constant [S7].

Four takeaways went up at the start. Here is what follows from them, which is a different list, and the one I would rather you carried [speculative].

First. Ask of every fast call whether it reported acceptance or completion. Success means accepted, not done. That one question covers today's material, and it covers a message broker acknowledging a message for the same structural reason.

Second. No level reports which one holds your data, so you cannot reason your way to it [V16]. You have to make the crossings visible and count them. That is why I spent twenty minutes on a measurement before I drew a single diagram.

Third. When you say the data is written, say which level holds it. Your own buffer, the C library's buffer, the page cache, or the device cache [inference].

Fourth. Measure the machine in front of you. The one constant I could safely have quoted for sixteen years moved by a factor of sixteen in Python 3.14.

Thank you. The handout has all of this in prose with every source. Questions now, and I am here afterwards for anything that needs a terminal.

58 58. References

On screen:

- [S1] *strace(1)* — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>
- [S2] *write(2)* — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html>
- [S3] *fflush(3)* — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>
- [S4] *close(2)* — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>
- [S5] C99 draft, §7.19.3. <https://busybox.net/~landley/c99-draft.html>
- [S6] Built-in Functions — *open()* — Python documentation. <https://docs.python.org/3/library/functions.html#open>
- [S7] Increase `io.DEFAULT_BUFFER_SIZE` — CPython issue 117151. <https://github.com/python/cpython/issues/117151>

- **[S8]** PEP 3116 — New I/O (2007). <https://peps.python.org/pep-3116/>
- **[S9]** `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- **[S10]** NtTrace — Native API tracing for Windows — Roger Orr. <https://roger-orr.github.io/NtTrace/>
- **[S11]** End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984). <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- **[S12]** Hints for Computer System Design — Lampson (1983). <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>
- **[S13]** fsync — The Open Group Base Specifications. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>
- **[S14]** Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S15]** The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S16]** FlexSC: Flexible System Call Scheduling with Exception-Less System Calls — Soares and Stumm, OSDI 2010. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf
- **[S17]** `io.IOBase.flush()` — Python documentation. <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>
- **[S19]** Inside Native Applications — Russinovich, Microsoft Learn (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** `os` — Miscellaneous operating system interfaces, `os.write()` — Python documentation. <https://docs.python.org/3/library/os.html#os.write>
- **[S21]** `fread(3)`, which documents `fwrite(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[V1]** A system call is not a function call — the two-term cost model.
- **[V2]** A system call is not a function call — the measured footprint of one crossing.
- **[V3]** A system call is not a function call — `pread` and `pwrite` merge the seek.
- **[V4]** A Successful `write()` Is a Handoff, Not a Save — the idiom with no durability step.
- **[V5]** A Successful `write()` Is a Handoff, Not a Save — the device level is unobservable.

- **[V6]** *Standard output is buffered differently depending on where it points — CPython above libc.*
- **[V7]** *Standard output is buffered differently depending on where it points — the measured 4096-byte buffer.*
- **[V8]** *Windows and Linux place the stable syscall boundary at different layers — the nomination argument.*
- **[V9]** *Windows and Linux place the stable syscall boundary at different layers — Linux as the outlier.*
- **[V10]** *The System Call Boundary — ltrace against strace.*
- **[V11]** *The System Call Boundary — io_uring with a polling thread.*
- **[V12]** *Where the Abstraction Leaks — accepted, not done.*
- **[V13]** *Where the Abstraction Leaks — trust the counts, not the times.*
- **[V14]** *Where the Abstraction Leaks — a kill is not a power cut.*
- **[V15]** *Where the Abstraction Leaks — a privilege transition, not a jump.*
- **[V16]** *Where the Abstraction Leaks — count, do not deduce.*
- **[V17]** *Observing a Running System — the 442-times worst case.*
- **[V18]** *The Memory Hierarchy from Python to Physical Pages — wholesale down, retail up.*
- **[V19]** *Virtual Memory and the Page Cache — name the cache level in every number.*
- **[V20]** *File IO and Durability — the device acknowledges early.*
- **[V21]** *NtTrace — Native API tracing for Windows (Roger Orr) — a debugger, not a driver.*
- **[V22]** *NtTrace — Native API tracing for Windows (Roger Orr) — the object-manager path prefix.*
- **[V23]** *NtTrace — Native API tracing for Windows (Roger Orr) — what the published trace cannot do.*
- **[V24]** *PostgreSQL's fsync() surprise — never retry after fsync fails.*
- **[V25]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot — not for production.*
- **[V26]** *The System Call Boundary — the calls a tracer cannot see.*
- **[V27]** *A system call is not a function call — what merging the seek did to overall duration.*
- **[V28]** *Sockets and Network I-O — a Linux socket is a file descriptor, a Windows socket is not.*

- **[V29]** *Sockets and Network I-O — sockets, pipes and terminals are all file descriptors.*
- **[V30]** *File IO and Durability — block-layer tracing watches the kernel-to-device boundary.*
- **[V31]** *Observing a Running System — the published range for the same benchmark, on other machines.*
- **[V32]** *Virtual Memory and the Page Cache — the page cache is the machine's own memory.*
- **[V33]** *File IO and Durability — the kernel issues the device cache-flush command.*
- **[V34]** *Virtual Memory and the Page Cache — readahead moves data up the same four levels.*
- **[V35]** *NtTrace — Native API tracing for Windows (Roger Orr) — the published trace is 32-bit and XP-era.*
- **[V36]** *Windows and Linux place the stable syscall boundary at different layers — the five systems do not decline the promise equally.*
- **[V37]** *File IO and Durability — FlushFileBuffers is the Windows counterpart of fsync.*
- **[V38]** *Windows and Linux place the stable syscall boundary at different layers — why Linux could not move its own boundary.*
- **[V39]** *Every Tracing Mechanism Trades Observation Cost for a Blind Spot — what the cheaper seccomp-bpf mode costs.*
- **[V40]** *PostgreSQL's fsync() surprise — three engines dropped the retry in 2018.*
- **[V41]** *PostgreSQL's fsync() surprise — the three engines are independent of each other.*

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

Every source behind every claim in this hour, in the order the deck used them.

The S entries are external, and every one has a link you can follow today. Most are manual pages, language documentation and standards. For this material the primary source is short, free and usually clearer than any summary of it.

The V entries are notes from my own research vault. You cannot open those, so the handout carries a one-page summary of each together with every external source it draws on.